

Τεχνολογία Πολυμέσων

Αναφορά Εξαμηνιαίας Εργασίας

Ιωάννης Τσαντήλας, 03120883

Repository & Setup

Το σύνολο του κώδικα της εργασίας μπορεί να βρεθεί σε αυτό το [Github repository](#).

Η υλοποίηση του project συμπεριλαμβάνει συγκεκριμένα *.jar* αρχεία, τα οποία έχουν πολύ μεγάλο μέγεθος για να γίνουν push στο repository. Για αυτό το λόγο, έχω δημιουργήσει ένα αρχείο README στο οποίο δίνω οδηγίες για να γίνει setup το project.

Βάση Δεδομένων

Η βάση δεδομένων μου αποτελείται από τέσσερα *.json* αρχεία στην διεύθυνση **multimedia24-ntua/multimedia/**. Συγκεκριμένα, αυτά τα αρχεία είναι:

- *categories.json*: αναλαμβάνει την αποθήκευση των κατηγοριών. Οι κατηγορίες έχουν τα εξής πεδία: *name*.
- *priorities.json*: αναλαμβάνει την αποθήκευση των προτεραιοτήτων. Οι προτεραιότητες έχουν τα εξής πεδία: *name*.
- *reminders.json*: αναλαμβάνει την αποθήκευση των υπενθυμίσεων. Οι υπενθυμίσεις έχουν τα εξής πεδία: *taskTitle*, *date*.
- *tasks.json*: αναλαμβάνει την αποθήκευση των εργασιών. Οι εργασίες έχουν τα εξής πεδία: *title*, *description*, *category*, *priority*, *deadline*, *status*.

Δομή – Παραδοχές

Η δομή του project χαρακτηρίζεται από τέσσερις αντίστοιχες οντότητες (entities): Εργασία (Task), Κατηγορία (Category), Προτεραιότητα (Priority) και Υπενθύμιση (Reminder).

Εργασία – Task

- Οι υποχρεωτικές πληροφορίες που πρέπει να δώσει ένας χρήστης για μία εργασία είναι ο τίτλος της, *title*, και η προθεσμία της, *deadline*.
- Ο τίτλος κάθε εργασίας θα πρέπει να είναι μοναδικός (not case-sensitive).
- Η προθεσμία δεν μπορεί να είναι παρελθοντική ημερομηνία.
- Σε περίπτωση που δεν δοθεί η:
 - Περιγραφή, *description*, αυτή θα τεθεί κενή.
 - Κατηγορία, *category*, αυτή θα τεθεί στην προκαθορισμένη τιμή της, Default.
 - Προτεραιότητα, *priority*, αυτή θα τεθεί στην προκαθορισμένη τιμή της, Default.
 - Κατάσταση, *status*, αυτό θα τεθεί στην προκαθορισμένη τιμή του, Open.
 - Σε αντίθεση με την κατηγορία και την προτεραιότητα, ο χρήστης δεν μπορεί να επεξεργαστεί τις καταστάσεις. Μπορεί να επιλέξει μόνο από τις διαθέσιμες: *Open*, *In Progress*, *Completed*, *Delayed*, *Postponed*.
 - Έτσι, σε περίπτωση που ο χρήστης βάλει κατάσταση *Delayed* σε μία εργασία με προθεσμία εντός 7 ημερών, αυτή η εργασία θα μετρηθεί και στα “*Delayed Tasks*” και στα “*Tasks due in 7 days*”.

Κατηγορία & Προτεραιότητα – Category & Priority

- Οι υποχρεωτικές πληροφορίες που πρέπει να δώσει ένας χρήστης για μία Κατηγορία/ Προτεραιότητα είναι το όνομα τους, *name*.
- Το όνομα κάθε Κατηγορίας/ Προτεραιότητας θα πρέπει να είναι μοναδικό (not case-sensitive).
- Ο χρήστης δεν μπορεί να διαγράψει ή να επεξεργαστεί την προκαθορισμένη Κατηγορία/ Προτεραιότητα, *Default*.
- Αν στο launch της εφαρμογής η Κατηγορία/ Προτεραιότητα *Default* δεν υπάρχει στο categories.json/ priorities.json, η εφαρμογή την δημιουργεί αυτόματα.

Υπενθύμιση – Reminder

- Οι υποχρεωτικές πληροφορίες που πρέπει να δώσει ένας χρήστης για μία Υπενθύμιση είναι η εργασία, *taskTitle*, και η ημερομηνία, *date*.
- Επειδή η υπενθύμιση ρυθμίζεται σε επίπεδο ημέρας, και όχι ώρας, για την ίδια εργασία, δεν μπορούν να υπάρχουν πολλές υπενθυμίσεις την ίδια ημέρα.
- Η ημερομηνία της υπενθύμισης δεν μπορεί να είναι πριν από την τωρινή ημερομηνία, ούτε μετά την προθεσμία της επιλεγμένης εργασίας.
 - Αυτό σημαίνει πως αν ο χρήστης επιλέξει μία προκαθορισμένη επιλογή π.χ. «μία εβδομάδα πριν την προθεσμία» και αυτή πέφτει πριν από την τωρινή ημερομηνία, ένα παράθυρο σφάλματος θα εμφανιστεί.
 - Φυσικά, το ίδιο θα συμβεί σε περίπτωση που ο χρήστης επιλέξει μία οποιαδήποτε ημερομηνία και αυτή πέφτει εκτός των επιτρεπτών ορίων.
- Εάν ο χρήστης αλλάξει την προθεσμία μία εργασίας, για την οποία κάποια υπενθύμιση έχει ημερομηνία μετά την νέα προθεσμία, η εφαρμογή ειδοποιεί τον χρήστη πως αν προχωρήσει, αυτή η υπενθύμιση θα διαγραφθεί.
 - Για παράδειγμα, έστω μία εργασία με προθεσμία 20/03/25 και υπενθύμιση 15/03/25. Εάν ο χρήστης αλλάξει την προθεσμία σε 10/03/25, η υπενθύμιση θα διαγραφθεί, αφού είναι εκτός ορίων.
- Εάν ο χρήστης αλλάξει την προθεσμία μία εργασίας, για την οποία κάποια υπενθύμιση έχει ημερομηνία πριν την νέα προθεσμία και την έχει επιλέξει μέσω των προκαθορισμένων επιλογών, η εφαρμογή αυτόματα ανανεώνει την ημερομηνία.
 - Για παράδειγμα, έστω μία εργασία με προθεσμία 20/03/25 και υπενθύμιση 13/03/25 – μέσω της επιλογής «1 εβδομάδα πριν». Εάν ο χρήστης αλλάξει την προθεσμία σε 19/03/25, η υπενθύμιση θα αλλάξει σε 12/03/25.
- Υπάρχει μία υποπερίπτωση (της υποπερίπτωσης...) σε αυτόν τον κανόνα. Εάν η προθεσμία είναι με μία προκαθορισμένη επιλογή, υπάρχει πιθανότητα η νέα ημερομηνία της υπενθύμισης να είναι στο παρελθόν. Σε αυτή την περίπτωση, ο χρήστης ειδοποιείται με αντίστοιχο παράθυρο και παροτρύνεται να ελέγξει τις υπενθυμίσεις του.
 - Για παράδειγμα, έστω πως σήμερα είναι 10/02/25. Έστω μία εργασία με προθεσμία 20/03/25 και υπενθύμιση 20/02/25 – μέσω της επιλογής «1 μήνας πριν». Εάν ο χρήστης αλλάξει την προθεσμία σε 09/03/25, τότε η υπενθύμιση θα αλλάξει σε 09/02/25, που είναι στο παρελθόν. Η υπενθύμιση αυτή θα παραμείνει, και ο χρήστης θα παροτρυνθεί να ελέγξει τις υπενθυμίσεις του.

Υλοποίηση

Η υλοποίηση ακολούθησε τους κανόνες του αντικειμενοστραφούς προγραμματισμού, διαχωρίζοντας ξεχωριστές κλάσεις, καθεμία από τις οποίες ενσωματώνει τα δικά της δεδομένα και συμπεριφορά.

Όπως φαίνεται και στην δομή του κώδικα εντός του **/multimedia24-ntua/src/**, η υλοποίηση ακολουθεί το [MVC μοντέλο](#). Τα μοντέλα (model) χειρίζονται τα δεδομένα, οι προβολές (views) την παρουσίαση (UI) και οι ελεγκτές (controllers) την αλληλεπίδραση μεταξύ των δύο.

- Οι κλάσεις μοντέλων (models) – όπως Task, Category, Priority και Reminder – περιβάλλουν χαρακτηριστικά με ιδιωτικά πεδία και εκθέτουν τη λειτουργικότητα μέσω getters και setters, εξασφαλίζοντας την ακεραιότητα των δεδομένων (data integrity) και την ενθυλάκωση (encapsulation).
- Οι ελεγκτές (controllers), όπως οι TaskController και CategoryController, διαχειρίζονται την επιχειρησιακή λογική και συντονίζουν τις ενέργειες του χρήστη χωρίς να εκθέτουν εσωτερικές υλοποιήσεις, γεγονός που ενισχύει την Αρχή της Ενιαίας Ευθύνης ([Single Responsibility Principle](#)). Επιπλέον, υλοποιούν επικυρώσεις εισόδου και ρουτίνες ενημέρωσης (π.χ. έλεγχος για διπλές εργασίες ή ενημέρωση της κατάστασης υπενθύμισης).
- Η χρήση του JavaFX για το μέρος της προβολής (views), με κλάσεις όπως οι TaskView και MainView, δείχνει πώς η διεπαφή χρήστη διατηρείται διακριτή από την υποκείμενη λογική δεδομένων.

Επιπλέον, η κλάση DataStore υλοποιεί το μοτίβο Singleton ([singleton pattern](#)), διασφαλίζοντας ότι όλα τα μέρη της εφαρμογής έχουν πρόσβαση σε μια ενιαία, συνεπή πηγή δεδομένων. Ο πολυμορφισμός ([polymorphism](#)) αξιοποιείται μέσω της δυναμικής συμπεριφοράς των UI στοιχείων, όπως κελιά που κάνουν override την μέθοδο updateItem. Η αφαίρεση (abstraction) είναι εμφανής σε μεθόδους όπως οι loadAllData() και saveAllData(), οι οποίες κρύβουν την πολυπλοκότητα της σειριοποίησης JSON (JSON serialization) χρησιμοποιώντας το Jackson.

Η ανάκτηση και ανανέωση των δεδομένων της εφαρμογής έγινε με βάση τις οδηγίες που δόθηκαν. Στην αρχικοποίηση της εφαρμογής γίνονται οι κατάλληλοι έλεγχοι (πχ. να ενημερωθεί ο χρήστης αν υπάρχουν *Delayed* εργασίες, αν υπάρχουν η προκαθορισμένη κατηγορία και προτεραιότητα, αν υπάρχουν εργασίες με παρελθοντική προθεσμία κα.), και με κάθε νέα αλλαγή γίνεται ανανέωση της βάσης δεδομένων μόλις ο χρήστης πατήσει “Save” (πχ. δημιουργία νέας εργασίας, διαγραφή κατηγορίας, επεξεργασία προτεραιότητας κα.).

Javadoc Documentation

Όλες οι κλάσεις εντός του **/multimedia24-ntua/src/** περιγράφηκαν σε Javadoc. Παρόλα αυτά, προτείνω το **/multimedia24-ntua/src/controllers/MainController.java** καθώς έχει περισσότερες public κλάσεις.

Για να δείτε αναλυτικά όλες τις κλάσεις, μπορείτε να ανοίξετε το αρχείο **/multimedia24-ntua/docs/index.html** στον αγαπημένο σας browser.

Σε περίπτωση που υπάρχει πρόβλημα με το αρχείο, ανατρέξτε στο README.md του repository, στην ενότητα *Javadoc Documentation*.